

TÀI LIỆU HƯỚNG DẪN NGHIỆP VỤ

7 SERVICE CỦA SMART CAMPUS OPERATIONS PLATFORM

Học phần FIT4110 — Dịch vụ kết nối và Công nghệ nền tảng

I. Mục tiêu và nguyên tắc nghiệp vụ

I.1. Vấn đề cần khắc phục

Qua quá trình theo dõi, nhận thấy phần lớn các nhóm đã hoàn thành phần kết nối kỹ thuật: subscribe được topic MQTT, đọc được stream camera, gọi được REST API giữa các service. Tuy nhiên, nhiều nhóm dừng lại ở mức trung chuyển dữ liệu: nhận gói tin rồi chuyển tiếp gần như nguyên trạng, không thực hiện xử lý nghiệp vụ.

Tài liệu này quy định rõ phần nghiệp vụ bắt buộc của từng service: dữ liệu nhận vào phải được kiểm tra, chuẩn hóa, phân loại, làm giàu và ra quyết định như thế nào trước khi gửi đi.

I.2. Nguyên tắc bắt buộc

Một service chỉ nhận dữ liệu và chuyển tiếp nguyên trạng được xem là chưa hoàn thành nhiệm vụ nghiệp vụ. Mỗi service phải thực hiện tối thiểu các nhóm thao tác sau đối với dữ liệu:

Thao tác	Yêu cầu
VALIDATE	Kiểm tra tính hợp lệ: đủ field bắt buộc, đúng kiểu dữ liệu, giá trị nằm trong miền cho phép
NORMALIZE	Chuẩn hóa định dạng, đơn vị, thời gian theo chuẩn thống nhất
ENRICH	Bổ sung thông tin từ nguồn tra cứu (whitelist, registry) mà payload thô không có
CLASSIFY / DECIDE	Áp quy tắc nghiệp vụ để phân loại trạng thái hoặc ra quyết định
PRODUCE	Tạo event đầu ra theo schema thống nhất, loại bỏ field nội bộ/debug

Tiêu chí tự kiểm tra: Nếu loại bỏ service của nhóm khỏi hệ thống mà các service phía sau vẫn nhận được dữ liệu đầy đủ và đúng nghiệp vụ, thì service của nhóm chưa tạo ra giá trị. Khi đó cần xem lại phần xử lý.

I.3. Phân biệt dữ liệu thô và sự kiện đã xử lý

Toàn hệ thống phân tách rõ hai loại topic:

- Topic raw (ví dụ smart-campus/raw/iot/environment): dữ liệu thô do simulator của giảng viên phát ra. Dữ liệu này chưa có kết luận nghiệp vụ, có thể chứa lỗi, thiếu field, hoặc thiết bị lạ.
- Topic events (ví dụ smart-campus/events/sensor): sự kiện đã được service của nhóm xử lý, làm sạch và bổ sung kết luận nghiệp vụ. Đây là đầu vào cho các service phía sau.

Nhiệm vụ cốt lõi của các nhóm tiếp nhận dữ liệu thô (IoT, Access) chính là biến topic raw thành topic events.

II. Kiến trúc kết nối toàn hệ thống

Sơ đồ phụ thuộc dưới đây quy định nhóm nào gọi nhóm nào, với mục đích gì và bằng cơ chế nào. Các nhóm phải tuân thủ đúng cơ chế kết nối đã quy định.

II.1. Bảng phụ thuộc giữa các service

Bên gọi (Consumer)	Bên cung cấp (Provider)	Mục đích	Cơ chế
Camera Stream	AI Vision	Gửi frame khi phát hiện chuyển động	REST sync
Core Business	AI Vision	Lấy kết quả phân tích ảnh / detect	REST sync
Core Business	Access Gate	Nhận log quét thẻ / kiểm tra quyền	REST sync
Access Gate	Core Business	Kiểm tra policy ra/vào theo thời gian thực	REST sync
Core Business	Notification	Kích hoạt gửi cảnh báo đa kênh	Queue async (MQTT)
IoT Ingestion	Core Business	Cấp event cảm biến đã xử lý	Queue async (MQTT)
IoT Ingestion	Analytics	Cấp telemetry để tổng hợp	Queue async (MQTT)
Camera Stream	Analytics	Cấp event camera để tổng hợp	Queue async (MQTT)
Core Business	Analytics	Cấp event cảnh báo / policy cho KPI	Queue async (MQTT)
Access Gate	Analytics	Cấp log ra/vào để thống kê	Queue async (MQTT)

II.2. Nguyên tắc chọn cơ chế kết nối

Hệ thống dùng hai cơ chế, mỗi cơ chế cho một mục đích khác nhau:

Cơ chế	Dùng khi nào
REST sync (gọi và chờ trả lời)	Khi bên gọi cần kết quả NGAY để xử lý tiếp: Camera hỏi AI Vision có gì trong ảnh; Core hỏi AI Vision/Access để ra quyết định realtime. Bên gọi dừng lại chờ phản hồi.
Queue async qua MQTT (gửi và đi tiếp)	Khi bên gửi chỉ cần phát sự kiện đi mà không cần chờ: IoT/Access/Camera/Core đẩy event lên topic; Analytics và Notification tự tiêu thụ. Bên gửi không bị chặn.

Ghi chú: Thông tin kết nối chi tiết (broker, port, username, password, tên topic) nằm trong README kết nối riêng của từng nhóm. Tài liệu này không lặp lại credential; các nhóm lấy từ README và đưa vào file .env theo đúng quy định bảo mật.

III. Phân vai nghiệp vụ 7 service

Service	Đầu vào chính	Trách nhiệm nghiệp vụ cốt lõi
IoT Ingestion	MQTT raw cảm biến	Làm sạch + phân loại trạng thái môi trường (normal/warning/danger)
Access Gate	MQTT raw UID thẻ	Đối chiếu danh tính + quyết định cho phép/từ chối ra vào
Camera Stream	MJPEG stream	Lọc khoảnh khắc có chuyển động + gọi AI Vision đúng lúc
AI Vision	REST nhận ảnh	Phân tích ảnh + đánh giá mức rủi ro, trả kết quả có cấu trúc
Core Business	MQTT events + REST	Áp policy, tổng hợp nhiều nguồn, ra quyết định cảnh báo
Notification	MQTT alert từ Core	Định tuyến cảnh báo đến đúng người, đúng kênh, có retry
Analytics	MQTT mọi events	Tổng hợp KPI và xu hướng cho dashboard

1. IoT Ingestion Service

Cửa ngõ chất lượng dữ liệu môi trường: biến tín hiệu cảm biến thô thành sự kiện sạch, có kết luận.

Vị trí trong kiến trúc

- Nhận: dữ liệu thô từ simulator của giảng viên qua MQTT (topic raw, chu kỳ 5 giây/gói).
- Gửi đi: Core Business và Analytics qua MQTT (Queue async) — theo Dependency Map mục II.1.
- Không gọi REST đồng bộ với service nào; vai trò của nhóm là nguồn phát sự kiện đã xử lý.

Đầu vào

Subscribe topic raw: smart-campus/raw/iot/environment.

Mỗi gói là một lần lấy mẫu môi trường, có thể chứa lỗi: giá trị null, thiết bị không đăng ký, giá trị vượt ngưỡng.

Schema payload nhận vào:

```
{
  "event_id": "raw-iot-abc123",
  "device_id": "esp32-lab-a101",
  "timestamp": "2026-06-07T14:30:10+07:00",
  "location": "Lab A101",
  "temperature_c": 31.2,
  "humidity_percent": 68.5,
  "motion_detected": false,
  "co2_ppm": 650,
  "smoke_ppm": 0.02,
  "battery_percent": 87,
  "scenario hint for teacher": "normal" // CHỈ để GV chấm - KHÔNG dùng trong logic
}
```

Yêu cầu xử lý bắt buộc

1. **VALIDATE schema:** kiểm tra đủ các field bắt buộc (event_id, event_type, timestamp, device_id, temperature_c, humidity_percent, motion_detected). Thiếu field bắt buộc thì ghi log lỗi và KHÔNG publish event sai schema.
2. **CHECK thiết bị:** đối chiếu device_id với file device_registry.csv. Thiết bị không có trong registry thì gán status = invalid_device, alert_level = high.
3. **NORMALIZE:** thống nhất đơn vị (nhiệt độ Celsius, độ ẩm phần trăm), kiểm tra timestamp đúng ISO 8601, ép kiểu số/boolean khi cần.
4. **CLASSIFY:** áp bộ quy tắc ngưỡng để xác định status (normal/warning/danger/sensor_error/invalid_device) và alert_level.
5. **PRODUCE:** tạo event mới theo schema thống nhất, LOẠI BỎ field scenario_hint_for_teacher, publish sang topic smart-campus/events/sensor.

Quy tắc nghiệp vụ

- sensor_error: temperature_c hoặc humidity_percent = null, hoặc dữ liệu sai kiểu số.
- invalid_device: device_id không có trong device_registry.csv.
- danger: temperature_c $\geq$ 40, hoặc co2_ppm $\geq$ 1800, hoặc smoke_ppm $\geq$ 1.0.
- warning: temperature_c $\geq$ 35, hoặc humidity_percent $\geq$ 85, hoặc co2_ppm $\geq$ 1200, hoặc smoke_ppm $\geq$ 0.5, hoặc battery_percent $<$ 20.
- normal: không rơi vào các trường hợp trên.

Đầu ra

Publish topic: smart-campus/events/sensor. Được Core Business và Analytics tiêu thụ.

Schema event gửi đi:

```
{
  "event_type": "sensor.reading.processed",
  "source_service": "team-iot",
  "raw_event_id": "raw-iot-abc123",
  "device_id": "esp32-lab-a101",
  "location": "Lab A101",
  "temperature_c": 42.1,
  "humidity_percent": 71.2,
  "motion_detected": true,
  "co2_ppm": 710,
  "smoke_ppm": 0.03,
  "battery_percent": 86,
  "status": "danger",
  "alert_level": "high",
  "reason": "temperature_too_high"
}
```

Nhóm phải tự quyết định

- Cách tổ chức đọc device_registry.csv (load lúc khởi động, không hard-code danh sách trong logic chính).
- Cách xử lý khi nhiều field cùng vượt ngưỡng (chọn mức cao nhất, hay ghi nhiều reason).
- Có giảm tần suất lưu/gửi khi dữ liệu liên tục bình thường không.
- Định nghĩa reason cụ thể cho từng status.

Lỗi thường gặp và điều cấm

- Dùng field scenario_hint_for_teacher để quyết định status/alert_level — vi phạm, bị trừ điểm.
- Publish nguyên payload raw sang topic events mà không phân loại — không đạt nghiệp vụ.
- Không kiểm tra device_registry — thiết bị lạ vẫn lọt vào hệ thống.
- Bỏ qua trường hợp null làm service crash thay vì gán sensor_error.

2. Access Gate Service

Người gác cổng: đối chiếu danh tính từ UID thẻ và quyết định cho phép hay từ chối ra vào.

Vị trí trong kiến trúc

- Nhận: UID thẻ thô từ simulator qua MQTT (topic raw, chu kỳ 10 giây/lần quét).
- Gửi đi: Analytics qua MQTT (Queue async) để thống kê log ra/vào.
- Cung cấp REST cho Core Business: Core gọi để kiểm tra policy ra/vào theo thời gian thực (REST sync) — theo Dependency Map.

Đầu vào

Subscribe topic raw: smart-campus/raw/access/rfid-uid.

Payload thô chỉ chứa UID, KHÔNG có tên sinh viên, KHÔNG có kết quả access. Nhóm phải tự tra cứu và quyết định.

Schema payload nhận vào:

```
{
  "event_id": "raw-rfid-abc123",
  "device_id": "rfid-reader-gate-01",
  "timestamp": "2026-06-07T14:30:10+07:00",
  "uid": "04:A1:B2:C3:D4:03",
  "door_id": "gate-a",
  "location": "Main Gate",
  "direction": "A",
  "in": "in"
}
```

Yêu cầu xử lý bắt buộc

6. VALIDATE: kiểm tra đủ field bắt buộc (event_id, event_type, timestamp, uid, door_id, direction).
7. LOOKUP: đối chiếu uid với file uid_whitelist.csv (load lúc khởi động, không hard-code danh sách).
8. DECIDE: nếu UID tồn tại thì access_result = granted, reason = uid_matched; nếu không tồn tại thì access_result = denied, reason = uid_not_found.
9. ENRICH: khi granted, gắn thêm student_id, full_name, class_name từ whitelist. Khi denied, các field này để null.
10. PRODUCE: publish event đã xử lý sang topic smart-campus/events/access. Ghi log mọi lượt (kể cả denied).

Quy tắc nghiệp vụ

- UID có trong uid_whitelist.csv → granted, gắn đầy đủ thông tin sinh viên.
- UID không có trong whitelist → denied, reason = uid_not_found, thông tin sinh viên = null.
- Mọi lượt quét đều phải được ghi log để Analytics thống kê và để truy vết.
- Khi Core gọi REST kiểm tra policy realtime, trả về kết quả granted/denied kèm lý do.

Đầu ra

Publish topic: smart-campus/events/access. Được Analytics (và Core) tiêu thụ.

Schema event gửi đi:

```
{
  "event_type": "access.swipe.processed",
  "source_service": "team-gate",
  "raw_event_id": "raw-rfid-abc123",
}
```

```

"uid": "04:A1:B2:C3:D4:03",
"student_id": "SV003",
"full_name": "Le Minh Cuong",
"class_name": "CNTT-K19",
"door_id": "gate-a",
"location": "Main Gate",
"direction": "in",
"access_result": "granted",
"reason": "uid_matched"
}

```

Nhóm phải tự quyết định

- Cách lưu và tra cứu uid_whitelist.csv (đọc file hay nạp vào database).
- Có bổ sung quy tắc theo giờ/khu vực không (ví dụ chặn vào khu hạn chế ngoài giờ) — nâng cao.
- Định nghĩa chuỗi quét bất thường (một UID quét nhiều lần liên tục).
- Định dạng REST trả về cho Core khi kiểm tra policy realtime.

Lỗi thường gặp và điều cấm

- Luôn trả granted mà không đối chiếu whitelist — không có nghiệp vụ.
- Không ghi log lượt denied — mất khả năng truy vết và thống kê.
- Publish payload thô (chỉ có UID) sang topic events mà không enrich danh tính.
- Hard-code danh sách UID trong code thay vì đọc từ file.

3. Camera Stream Service

Bộ lọc khoảng khắc: chọn frame có chuyển động đáng phân tích và gọi AI Vision đúng lúc, tránh quá tải.

Vị trí trong kiến trúc

- Nhận: luồng MJPEG từ URL camera dùng chung do giảng viên cấp (đọc frame qua HTTP).
- Gọi AI Vision qua REST (REST sync): gửi frame khi phát hiện chuyển động, chờ kết quả detect.
- Gửi Analytics qua MQTT (Queue async): cấp event camera để tổng hợp.
- Có thể gửi event sang Core sau khi có kết quả AI, tùy contract liên nhóm.

Đầu vào

Đọc luồng MJPEG từ URL stream (xem README kết nối). Stream trả về chuỗi frame liên tục, không kèm metadata.

Nhóm phải tự gắn metadata cho frame: frame_id, camera_id, timestamp, location, kích thước.

Yêu cầu xử lý bắt buộc

11. CHECK kết nối: kiểm tra đọc được frame, frame không rỗng, kích thước hợp lệ, stream không timeout. Lỗi thì log rõ ràng (camera_stream_unavailable).
12. DETECT motion: chuyển frame sang grayscale, so sánh với frame trước (frame difference), tính độ khác biệt, so với ngưỡng.
13. THROTTLE: không gửi mọi frame. Chỉ gửi khi có chuyển động, áp cooldown 5-10 giây sau mỗi lần gọi AI.
14. PREPROCESS: resize về kích thước hợp lý (ví dụ 640x480), nén JPEG quality 70-85, loại frame đen/rỗng/quá lớn.
15. CALL AI Vision: đóng gói request đúng schema thống nhất với nhóm AI Vision (ưu tiên gửi snapshot_url khi chạy nội bộ để tránh payload lớn), gọi REST.
16. HANDLE kết quả + lỗi: xử lý timeout (retry tối đa 2 lần), AI trả 500, AI không reachable. Không để service crash.

Quy tắc nghiệp vụ

- motion_detected = false → KHÔNG gọi AI Vision.
- motion_detected = true → gửi 1 snapshot sang AI Vision, sau đó chờ hết cooldown mới gửi tiếp.
- Tần suất gọi AI tối đa: tham khảo 1 frame mỗi 1-3 giây cho mỗi camera.
- AI trả risk_level cao / unknown_person = true → gửi event sang Core (hoặc theo contract).

Đầu ra

REST request sang AI Vision (POST /api/v1/detect).

MQTT publish smart-campus/events/camera để Analytics tổng hợp.

Schema event gửi đi:

```
{
  "request_id": "vision-request-001",
  "event_type": "camera.motion.triggered",
  "source_service": "team-camera",
  "camera_id": "cam-gate-a",
  "timestamp": "2026-06-07T14:30:10+07:00",
}
```

```
"location": "Main" "Gate" "A",
"motion_detected": true,
"motion_score": 0.82,
"snapshot_url": "http://team-camera/snapshots/cam-gate-a/20260607_143010.jpg"
}
```

Nhóm phải tự quyết định

- Ngưỡng motion_score để coi là có chuyển động.
- Chọn gửi image_base64 hay snapshot_url (khuyến nghị snapshot_url khi chạy Docker nội bộ).
- Thời gian cooldown sau mỗi lần gọi AI.
- Cách lưu và đặt tên snapshot, thời gian giữ lại.

Lỗi thường gặp và điều cấm

- Gửi toàn bộ luồng video hoặc 30 frame/giây sang AI Vision — cấm.
- Không kiểm tra frame rỗng/đen trước khi gửi.
- Để service crash khi camera mất kết nối thay vì log và chờ.
- Gửi request thiếu timestamp, camera_id hoặc request_id.

4. AI Vision Service

Bộ phân tích hình ảnh: nhận ảnh, nhận diện đối tượng và đánh giá mức rủi ro, trả kết quả có cấu trúc.

Vị trí trong kiến trúc

- Nhận: REST request từ Camera Stream (gửi frame khi có motion) và từ Core Business (lấy kết quả phân tích) — cả hai đều REST sync.
- Trả về: kết quả detect đồng bộ trong response. AI Vision là service kiểu hỏi-đáp, không tự phát MQTT.
- Lưu ý schema dưới đây là đề xuất; nhóm phải thống nhất contract chính thức với Camera và Core.

Đầu vào

Endpoint REST: POST /api/v1/detect.

Nhận snapshot_url (hoặc image_base64) kèm metadata camera_id, request_id, timestamp.

Schema payload nhận vào:

```
{
  "request_id": "vision-request-001",
  "camera_id": "cam-gate-a",
  "timestamp": "2026-06-07T14:30:10+07:00",
  "location": "Main Gate A",
  "motion_score": 0.82,
  "snapshot_url": "http://team-camera/snapshots/cam-gate-a/20260607_143010.jpg"
}
```

Yêu cầu xử lý bắt buộc

17. RECEIVE: nhận request, tải ảnh từ snapshot_url hoặc giải mã image_base64. Kiểm tra ảnh hợp lệ.
18. ANALYZE: chạy model (YOLOv8 thật hoặc mock có mô tả rõ): phát hiện đối tượng (person, vehicle...), số lượng, vị trí bbox.
19. SCORE: gán confidence cho mỗi phát hiện, chỉ giữ phát hiện đạt ngưỡng tin cậy.
20. ASSESS: đánh giá rủi ro dựa trên đối tượng và ngữ cảnh (giờ, vị trí), xác định unknown_person và risk_level.
21. RESPOND: trả kết quả có cấu trúc trong response REST, kèm request_id để bên gọi đối soát.

Quy tắc nghiệp vụ

- Chỉ giữ phát hiện có confidence ≥ 0.5 (ngưỡng do nhóm chốt).
- Phát hiện person + ngữ cảnh đáng ngờ (ngoài giờ / khu hạn chế) $\rightarrow$ risk_level cao, unknown_person = true.
- Không phát hiện gì $\rightarrow$ detections rỗng, risk_level thấp.
- Nếu dùng mock thay model thật: phải trả đúng cấu trúc và ghi rõ là mock trong tài liệu nhóm.

Đầu ra

REST response trả về cho Camera/Core.

Schema event gửi đi:

```
{
  "request_id": "vision-request-001",
  "camera_id": "cam-gate-a",
  "timestamp": "2026-06-07T14:30:11+07:00",
  "detections": [
```

```
{
  "label": "person",
  "confidence": 0.92,
  "bbox": {
    "x": 120,
    "y": 80,
    "width": 210,
    "height": 430
  }
},
"unknown person": true,
"risk_level": "warning"
}
```

Nhóm phải tự quyết định

- Loại đối tượng cần nhận diện (chỉ person hay nhiều loại).
- Ngưỡng confidence chấp nhận một phát hiện.
- Công thức tính risk_level từ đối tượng và ngữ cảnh.
- Dùng model thật hay mock, và logic của mock nếu dùng.

Lỗi thường gặp và điều cấm

- Chỉ trả có người / không người, không đánh giá rủi ro.
- Trả mọi phát hiện kể cả confidence rất thấp.
- Dùng mock nhưng trình bày như model thật — vi phạm trung thực học thuật.
- Không trả request_id, khiến bên gọi không đối soát được.

5. Core Business Service

Trung tâm ra quyết định: tổng hợp sự kiện từ nhiều nguồn, áp policy và quyết định khi nào phát cảnh báo.

Vị trí trong kiến trúc

- Nhận qua MQTT: event đã xử lý từ IoT (smart-campus/events/sensor), Access (smart-campus/events/access), Camera (smart-campus/events/camera).
- Gọi qua REST: AI Vision (lấy kết quả detect) và Access Gate (kiểm tra quyền) khi cần quyết định realtime.
- Gửi đi: Notification (kích hoạt cảnh báo) và Analytics (cấp event alert/policy) qua MQTT.
- Đây là service trung tâm và phức tạp nhất; phần lớn logic nghiệp vụ nằm ở đây.

Đầu vào

Subscribe đồng thời nhiều topic events từ IoT, Access, Camera.

Mỗi event đã được service nguồn phân loại sơ bộ (status, access_result, risk_level). Core áp policy cấp hệ thống lên các event này.

Yêu cầu xử lý bắt buộc

22. RECEIVE: tiêu thụ event từ các topic events. Có thể cần lưu tạm để kết hợp nhiều event trong một khung thời gian.
23. APPLY POLICY: áp bộ quy tắc nghiệp vụ. Quyết định event (hoặc tổ hợp event) có cấu thành sự cố cần cảnh báo không.
24. DECIDE severity: xác định mức nghiêm trọng (low/medium/high/critical) của cảnh báo.
25. CREATE ALERT: tạo bản ghi cảnh báo có định danh, loại, mức độ và nội dung rõ ràng.
26. DISPATCH: gửi cảnh báo sang Notification (MQTT) và cấp event alert/policy cho Analytics (MQTT).
Lưu audit để truy vết.

Quy tắc nghiệp vụ

- status = danger từ IoT (ví dụ smoke hoặc nhiệt độ vượt ngưỡng nguy hiểm) → tạo alert mức cao.
- Camera/AI báo unknown_person + Access báo bất thường cùng khu trong một khung thời gian ngắn → nghi đột nhập → alert cao.
- Access trả denied lặp lại nhiều lần → nghi dò thẻ → alert trung bình.
- motion_detected hoặc lượt ra/vào ngoài giờ cho phép → đánh giá theo policy giờ giấc.
- Event bình thường (status = normal, access = granted hợp lệ) → ghi nhận, không tạo alert.

Đầu ra

MQTT: gửi alert sang Notification; cấp event alert/policy cho Analytics.

Lưu audit log mọi quyết định.

Schema event gửi đi:

```
{
  "event type": "core.alert.created",
  "source service": "team-core",
  "alert_id": "ALT-001",
  "alert_type": "fire",
  "severity": "critical",
}
```

```
"message": "Phat hien khoi nong do cao tai Lab A101",
"origin_event_id": "sensor-event-002",
"target": "security_team",
"timestamp": "2026-06-07T14:30:17+07:00"
}
```

Nhóm phải tự quyết định

- Toàn bộ bộ luật: điều kiện nào tạo alert và ở mức nào (phần chất xám lớn nhất của hệ thống).
- Có kết hợp nhiều event không, trong khung thời gian bao lâu.
- Cơ chế chống cảnh báo trùng (cùng một sự cố không phát nhiều lần).
- Khi nào gọi AI Vision/Access bằng REST để bổ sung dữ liệu trước khi quyết định.

Lỗi thường gặp và điều cấm

- Nhận event rồi chuyển thẳng sang Notification mà không áp policy — Core trở thành ống dẫn.
- Tạo alert cho mọi event kể cả bình thường — gây nhiễu cảnh báo.
- Không lưu audit — không giải thích được vì sao có cảnh báo.
- Bộ luật chỉ có một điều kiện đơn giản — không thể hiện vai trò trung tâm.

6. Notification Service

Khâu phát cảnh báo: đảm bảo cảnh báo đến đúng người, đúng kênh, đúng mức ưu tiên và được gửi thành công.

Vị trí trong kiến trúc

- Nhận: cảnh báo từ Core Business qua MQTT (Queue async).
- Gửi đi: các kênh bên ngoài (Telegram, email, webhook). Là điểm cuối của luồng cảnh báo.
- Schema nhận vào theo event alert do Core phát; nhóm thống nhất contract với Core.

Đầu vào

Subscribe topic cảnh báo do Core phát (ví dụ smart-campus/events/alert hoặc topic thống nhất với Core).

Mỗi cảnh báo có severity, target, message và định danh.

Yêu cầu xử lý bắt buộc

27. RECEIVE: tiêu thụ cảnh báo từ Core.
28. ROUTE: xác định người nhận theo target và severity (security_team, admin, hoặc tất cả).
29. PICK CHANNEL: chọn kênh theo mức độ — critical gửi nhiều kênh đồng thời, mức thấp chỉ ghi log.
30. SEND: gửi thật qua kênh đã chọn.
31. RETRY + RECORD: nếu gửi lỗi thì thử lại theo chính sách; ghi lại trạng thái gửi (đã gửi, kênh, thành công/thất bại).

Quy tắc nghiệp vụ

- severity = critical → gửi ngay qua tối thiểu 2 kênh.
- severity = high → gửi cho đội phụ trách qua kênh tức thời.
- severity = medium → gửi qua kênh không gấp (ví dụ email).
- severity = low → chỉ ghi log, không làm phiền người dùng.
- Gửi lỗi → retry theo số lần và khoảng cách do nhóm chốt; có cơ chế chống gửi trùng.

Đầu ra

Tin nhắn thực tế ra các kênh.

Bản ghi trạng thái gửi cho mỗi cảnh báo.

Nhóm phải tự quyết định

- Bảng định tuyến: loại/mức cảnh báo → người nhận → kênh.
- Chính sách retry: số lần, khoảng cách.
- Cơ chế chống gửi trùng.
- Định dạng nội dung tin cho từng kênh.

Lỗi thường gặp và điều cấm

- Mọi cảnh báo gửi cùng một kênh, cùng mức ưu tiên.
- Gửi lỗi thì bỏ luôn, không retry — mất cảnh báo quan trọng.
- Chỉ in ra console mà không gửi thật.
- Không ghi trạng thái gửi.

7. Analytics Service

Bộ tổng hợp: biến hàng nghìn event rời rạc thành các chỉ số và xu hướng phục vụ quản lý.

Vị trí trong kiến trúc

- Nhận qua MQTT (Queue async): event từ IoT, Access, Camera và event alert/policy từ Core.
- Cung cấp: REST endpoint cho dashboard truy vấn KPI.
- Là điểm hội tụ dữ liệu của toàn hệ thống.

Đầu vào

Subscribe các topic events: smart-campus/events/sensor, smart-campus/events/access, smart-campus/events/camera và event cảnh báo từ Core.

Dữ liệu đến liên tục; Analytics tích lũy và tổng hợp theo thời gian, khu vực, loại sự kiện.

Yêu cầu xử lý bắt buộc

32. COLLECT: tiêu thụ event từ các topic, lưu trữ phục vụ tổng hợp.
33. AGGREGATE: tính tổng, trung bình, đếm theo các chiều (phòng, ngày, loại).
34. ANALYZE TREND: xác định xu hướng (giờ cao điểm, khu nhiều cảnh báo, thiết bị pin yếu).
35. STRUCTURE + SERVE: đóng gói KPI thành báo cáo gọn và cung cấp qua REST cho dashboard.

Quy tắc nghiệp vụ

- Tính nhiệt độ/độ ẩm trung bình theo phòng, không gộp chung toàn tòa nhà.
- Đếm số lần warning/danger theo ngày.
- Theo dõi thiết bị pin yếu theo device.
- Đếm cảnh báo phân theo severity và theo khu vực.
- Xác định giờ cao điểm ra/vào từ log Access.
- Visual dữ liệu lên giao diện.

Đầu ra

REST endpoint cung cấp KPI (ví dụ GET /metrics/daily).

Schema event gửi đi:

```
{
  "date": "2026-06-07",
  "avg_temperature_by_room": {
    "A101": 31.5,
    "B201": 28.2
  },
  "danger_event_count": 3,
  "warning_event_count": 12,
  "low_battery_device_count": 1,
  "total_access_in": 120,
  "denied_access count": 4,
  "peak access hour": "07:00-08:00"
}
```

Nhóm phải tự quyết định

- Bộ KPI quan trọng nhất cho người quản lý.
- Chiều tổng hợp (thời gian, khu vực, loại sự kiện).
- Tính realtime mỗi truy vấn hay tính sẵn định kỳ.
- Cấu trúc báo cáo để dashboard dễ vẽ biểu đồ.

Lỗi thường gặp và điều cấm

- Trả dữ liệu thô hàng nghìn dòng thay vì tổng hợp.
- Gộp mọi thứ thành một con số vô nghĩa, mất chiều thông tin.
- Lấy dữ liệu nhưng không tính xu hướng.
- Không phân tách theo phòng/khu/thời gian.

IV. Ba kịch bản nghiệp vụ xuyên suốt

Các kịch bản sau cho thấy vai trò của từng service trong một luồng hoàn chỉnh, theo đúng kiến trúc topic và cơ chế kết nối ở mục II.

Kịch bản 1 — Cảnh báo môi trường nguy hiểm

- IoT Ingestion nhận gói raw có smoke_ppm cao từ topic raw → validate → kiểm tra registry → phân loại status = danger → publish smart-campus/events/sensor.
- Core Business tiêu thụ event sensor → áp policy "danger = tạo cảnh báo" → tạo alert critical → gửi Notification (MQTT) và cập event cho Analytics (MQTT).
- Notification nhận alert critical → gửi tối thiểu 2 kênh cho đội phụ trách → retry nếu lỗi → ghi trạng thái.
- Analytics ghi nhận một sự kiện danger trong ngày cho KPI.

Kịch bản 2 — Phát hiện người lạ tại cổng

- Camera Stream đọc stream → phát hiện motion → tạo snapshot → gọi AI Vision qua REST.
- AI Vision phân tích ảnh → phát hiện person, unknown_person = true, risk_level cao → trả kết quả đồng bộ.
- Camera Stream publish event camera (MQTT) cho Analytics; gửi tín hiệu sang Core nếu rủi ro cao.
- Core Business áp policy → tạo alert → Notification gửi cảnh báo kèm thông tin.

Kịch bản 3 — Quẹt thẻ và kiểm tra quyền

- Access Gate nhận UID raw từ topic raw → đối chiếu uid_whitelist.csv → quyết định granted/denied → enrich danh tính → publish smart-campus/events/access.
- Core Business có thể gọi REST sang Access để kiểm tra policy ra/vào theo thời gian thực khi cần ra quyết định.
- Analytics tiêu thụ log access để thống kê lượt ra/vào và số lượt bị từ chối.
- Nếu phát hiện bất thường (denied lặp lại), Core tạo alert và chuyển Notification.

Bảng tổng kết giá trị nghiệp vụ và tiêu chí đánh giá

Service	Biến đổi nghiệp vụ	Tiêu chí đánh giá chính
IoT Ingestion	Dữ liệu cảm biến thô → event có status + alert_level	Có validate, check registry, phân loại đúng ngưỡng, loại field debug
Access Gate	UID thô → quyết định granted/denied + danh tính	Có đối chiếu whitelist, enrich, ghi log đầy đủ
Camera Stream	Luồng video → snapshot có motion gửi AI đúng lúc	Có lọc motion, throttle, xử lý lỗi AI
AI Vision	Ảnh → kết quả detect + đánh giá rủi ro	Có đánh giá risk_level, lọc theo confidence
Core Business	Nhiều event → quyết định cảnh báo theo policy	Có bộ luật thực chất, severity, audit, chống trùng
Notification	Cảnh báo → gửi đúng người/kênh có retry	Có định tuyến theo mức, retry, ghi trạng thái
Analytics	Nghìn event → KPI và xu hướng	Có tổng hợp đa chiều, không trả dữ liệu thô